accidentally remove the encryption if they simply commented-out the `EncryptedTokenStore` wrapper in the main method, losing both security properties. If you'd developed the `EncryptedTokenStore` using an unauthenticated encryption scheme like CTR mode and then manually combined it with the `HmacTokenStore` the risk would be even greater because not every way of combining those two stores is secure as you learnt in section 6.3.1.

The kit-of-parts approach to software design is often appealing to software engineers, because it results in a neat design with proper separation of concerns and maximum reusability. This was useful when you could reuse the `HmacTokenStore`, originally designed to protect database-backed tokens, to also protect JSON tokens stored on the client. But a kit-of-parts design is opposed to security if there are many insecure ways to combine the parts and only a few that are secure.

> **PRINCIPLE** Secure API design should make it very hard to write insecure code. It is not enough to merely make it possible to write secure code, because developers will make mistakes.

You can make a kit-of-parts design harder to misuse by using types to enforce the security properties you need, as shown in figure 6.6. Rather than all the individual token stores implementing a generic `TokenStore` interface, you can define *marker interfaces* that describe the security properties of the implementation. A `ConfidentialTokenStore` ensures that token state is kept secret, while an `AuthenticatedTokenStore` ensures that the token cannot be